

19 - Polymorphism & Dynamic Binding; Behavioral Replacement; Subtyping vs Inheritance

A. Theory (1:30)

- Compile-time vs run-time types; overriding vs overloading
- Dynamic dispatch: virtual methods; method binding vs field hiding
- Overriding rules: visibility, exceptions, covariant return types
- Effects of `final`, `static`, and `private` on dispatch; `@Override` safety

20 - Upcasting & Downcasting; `Object.equals()` Basics

A. Theory (1:30)

- Safe upcasts; downcast risks and `ClassCastException`
- Type tests: `instanceof` and pattern matching (where available); prefer polymorphic calls
- `equals()` contract: reflexive, symmetric, transitive, consistent, non-null
- `equals()` vs `==`; relation to `hashCode()` (consistency)
- Pitfalls: floating-point, arrays, collections, mutability leaks

21 - Abstract Classes & Abstract Methods

A. Theory (1:30)

- Abstract classes for shared state + partial implementation
- Abstract vs concrete members; constructors and initialization order
- Template Method idea; enforcing invariants via abstract hooks
- Choosing abstract class vs interface; evolution and binary compatibility
- (Optional) Sealed types overview for constrained hierarchies

22 - Interfaces; `Comparable` & `Cloneable`

A. Theory (1:30)

- Interfaces as capability contracts; multiple inheritance of type; interface inheritance
- Default and static methods; conflict resolution rules; private methods in interfaces
- Functional interfaces and `@FunctionalInterface`; core types (`Predicate`, `Function`, `Supplier`, `Consumer`, `Bi*`)
- Marker/tagging interfaces (`Serializable`, `Cloneable`) and design intent
- `Comparable<T>`: total ordering contract; consistency with `equals()`
- `Comparator<T>`: custom orderings; stability considerations
- Copy semantics: shallow vs deep copy; `Cloneable` caveats; safer copy patterns

23 - `ArrayList` Class

A. Theory (1:30)

- Dynamic array model: capacity vs size; growth policy; amortized analysis
- Operation costs: add, get, set, remove, contains; index vs iterator traversal
- Iterators: fail-fast behavior; concurrent modification caveats
- Views and copies: subList, List.copyOf, List.of immutability notes
- Equality, ordering, and sorting interactions with lists

24 - Generics; Static vs Dynamic Typing (in Java)

A. Theory (1:30)

- Generic classes/methods; type parameters; bounds and multiple bounds
- Wildcards and variance: ?, ? extends, ? super
- Type inference (diamond), raw types, bridge methods
- Type erasure: non-reifiable types, array limitations, reflection notes
- Static typing with late binding: compile-time safety vs run-time dispatch

25 - Introduction to File I/O (Text) - `java.io` + NIO.2 basics

A. Theory (1:30)

- Path and Files (NIO.2): path resolution, readString/writeString, lines, newBufferedReader/Writer
- Character sets and encodings (UTF-8); newline portability
- Try-with-resources; checked exceptions; error handling strategies
- Directory operations: create, walk (Files.walk), globbing (DirectoryStream with patterns)
- WatchService (intro) for directory change notifications
- File attributes/metadata: BasicFileAttributes, permissions, hidden/symlink notes

26 - Binary Files (Object Streaming Foundations) — Streams, Channels & Buffers

A. Theory (1:30)

- Streams vs Readers/Writers; buffering and EOF
- Primitive I/O with DataInput/DataOutput; record layout design; endianness/alignment
- NIO Channels & ByteBuffer: FileChannel, scattering/gathering reads/writes
- Memory-mapped files (FileChannel.map) and when to use them
- Asynchronous I/O: AsynchronousFileChannel (overview), thread pools, completion handlers

27 - Object Serialization

A. Theory (1:30)

- Serializable mechanism; serialVersionUID; transient fields
- Default vs custom serialization: writeObject/readObject, readResolve/writeReplace
- ObjectInputValidation; filters and deny/allow lists
- Security & performance concerns; versioning strategies
- Alternatives and migration: JSON/CBOR/ProtoBuf; schema evolution

28 - GUI Components (JavaFX): Label, Button, TextField, TextArea, RadioButton, ComboBox, ListView, ScrollBar

A. Theory (1:30)

- JavaFX app model: Application, Stage, Scene, scene graph
- Core controls and their models; observable properties and ObservableList
- CSS styling; themes; basic accessibility/i18n notes
- FXML basics and controller wiring; lifecycle hooks

29 - Containers & Layout Managers (JavaFX Layout Panes)

A. Theory (1:30)

- Layout panes: HBox, VBox, BorderPane, GridPane, StackPane, FlowPane, AnchorPane
- Sizing policy: preferred/min/max sizes; alignment, padding, spacing
- Responsive design: percentage-based layouts, DPI/scaling considerations
- Organizing complex UIs with nested panes; maintainability guidelines

30 - Event-Driven Programming; Events & Listeners (Intro) - JavaFX

A. Theory (1:30)

- Event model: Event, EventTarget, EventHandler; capture vs bubble phases; event filters vs handlers
- Common event types: action, input (key/mouse), selection, focus
- JavaFX Application Thread (JAT); Platform.runLater; concurrency cautions
- Background work: Task and Service (overview); progress and cancellation
- MVC/MVVM separation with properties and bindings

31 - Inner & Anonymous Classes; Lambdas for Event Handling

A. Theory (1:30)

- Member, local, and anonymous classes; scoping and shadowing rules
- Effectively-final capture; lifecycle and memory considerations
- Functional interfaces; lambda syntax; method references
- Differences between lambdas and anonymous classes (typing, identity)
- Best practices for concise, maintainable JavaFX event handlers

32 - Reactive-Style Updates; Externally-Generated Events; Window Events; (JavaFX equivalents of “adapter” patterns)

A. Theory (1:30)

- Property bindings: uni/bi-directional, binding expressions; ChangeListener vs InvalidListener
- Observable collections (ObservableList); automatic UI refresh patterns
- Stage/Window lifecycle: setOnCloseRequest, onShowing/Shown/Hiding/Hidden; graceful shutdown policies
- Timers and debouncing: Timeline, AnimationTimer, scheduled tasks; reacting to external triggers (WatchService, sockets)
- Related libraries: ControlsFX, RichTextFX (and optional RxJavaFX overview for reactive pipelines)